

CS330 Operating Systems and Lab.

Scheduling

Spring 2026

KAIST

Instructor : Insik Shin

Today: Concurrency – Scheduling

Goals:

- Multiprogramming: Run multiple applications concurrently

Challenges of Multiprogramming

- Applications compete with each other for resources
 - Need to arbitrate access to shared resources → **concurrency (scheduling & synchronization)**
 - Need to protect applications from each other → **protection**
- Applications need to communicate/cooperate with each other → **concurrency**

Today: Concurrency – Scheduling

Goals:

- Multiprogramming: Run multiple applications concurrently



Challenge:

- Executing multiple applications on a single resource



Solution:

Scheduling (multiplexing)

- Determine how to assign resource to applications (i.e., determining the order of execution) for some given objectives

Design Principles in Computer Science

- Abstraction
- Separation between policy and mechanism



Separation btw. Policy & Mechanism: Example

- Policy: who can enter a hotel room?
- Mechanism: how can we enforce the policy?
 - what if a policy changes in the middle?
 - what if any one does not follow rules?
- Card keys
 - Mechanism
 - Magnetic card readers, remote controlled locks, connections to a security server
 - Policy
 - Different guests can enter the room for different designated times (e.g., connecting rooms)
 - Hotel staff may enter a room at any time
- Physical keys
 - Changing who can open a door has to issue new keys and change the lock



Design Principles in Computer Science

- Abstraction
- Separation between policy and mechanism
 - Decoupling the mechanism implementations from the policy specifications makes it possible for different applications to use the same mechanism implementations with different policies.
 - This means that any **hard-coded policies** are likely to be inadequate or inappropriate for some (or perhaps even most) users

You can start by hard-coding to test the viability of various design concepts. Once proven, it would be wise to refine the designs by eliminating the hard-coded parts.

Scheduling

- Determines which processes make use of which resources
 - **Scheduler**
 - High-level **policy**
 - Responsibility: **Deciding which process to run**
 - **Dispatcher**
 - Low-level **mechanism**
 - Responsibility: **context switch**
 - Save previous process state in PCB
 - Load next process state from PCB to registers
 - Change scheduling state of process (running, ready, or blocked)
 - Migrate processes between different scheduling queues
 - Switch from kernel to user mode

Dispatch Loop

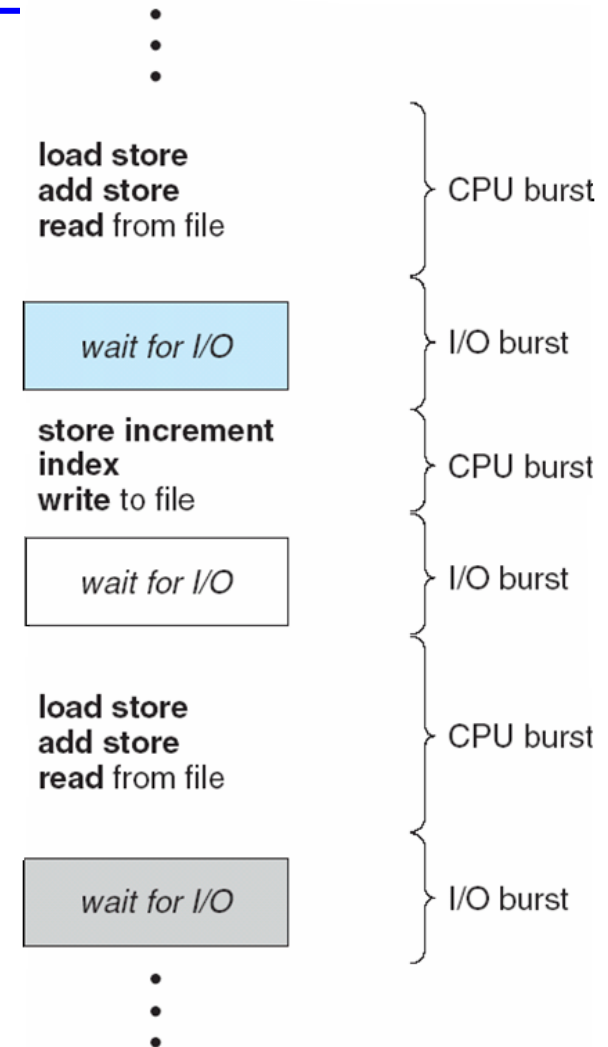
- Conceptually, the dispatching loop of the operating system looks as follows:

```
Loop {  
    RunThread();  
    ChooseNextThread();  
    SaveStateOfCPU(curTCB);  
    LoadStateOfCPU(newTCB);  
}
```

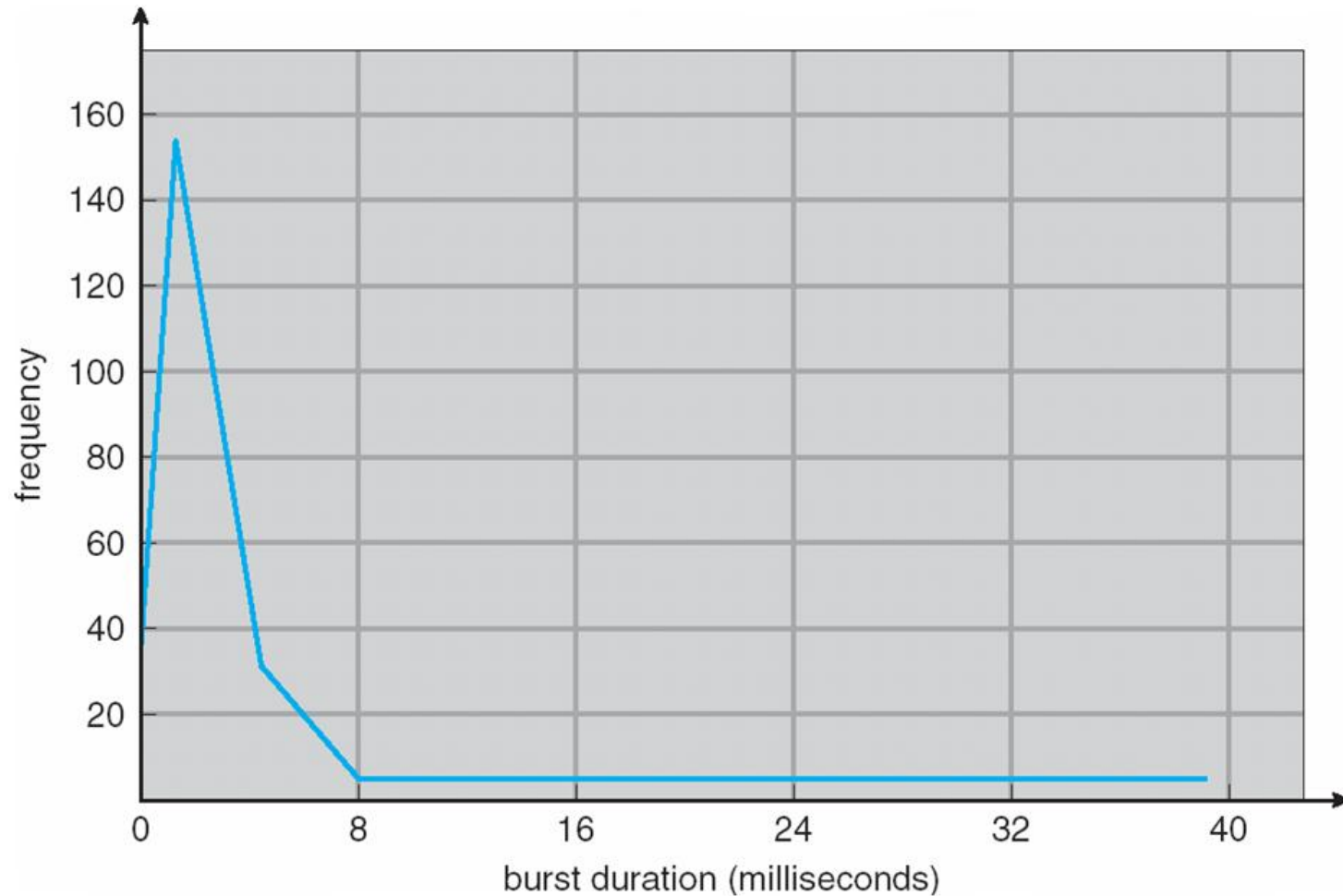
- This is an *infinite* loop
 - One could argue that this is all that the OS does

Background

- CPU–I/O Burst Cycle
 - Process execution consists of a *cycle* of CPU execution and I/O wait
- Multiprogramming
 - Allows to obtain maximum CPU utilization



Histogram of CPU-burst Times



Scheduling Criteria

- **CPU utilization**
 - keep the CPU as busy as possible
- **Throughput**
 - # of processes that complete their execution per time unit
- **Turnaround time**
 - amount of time to execute a particular process
- **Waiting time**
 - amount of time a process has been waiting in the ready queue
- **Response time**
 - amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

Scheduling Algorithms

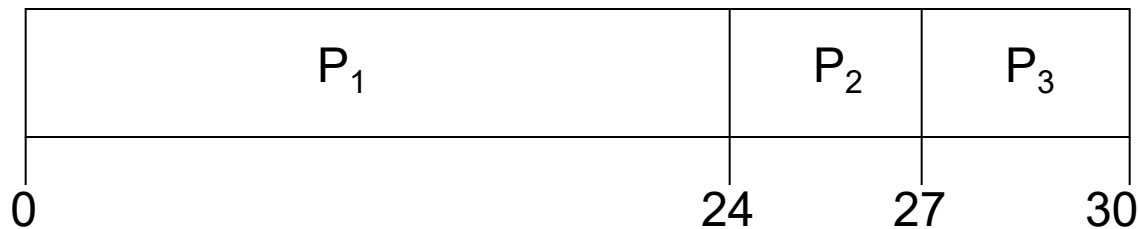
- Optimization Criteria
 - Max CPU utilization
 - Max throughput
 - Min turnaround time
 - Min waiting time
 - Min response time
- Scheduling algorithms
 - FCFS (first-come-first-serve)
 - SJF (shorted job first)
 - RR (round-robin)
 - Priority-based scheduling
 - Multi-level queue scheduling

First-Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3



- Processes arrive in the order: P_1 , P_2 , P_3
the schedule is:



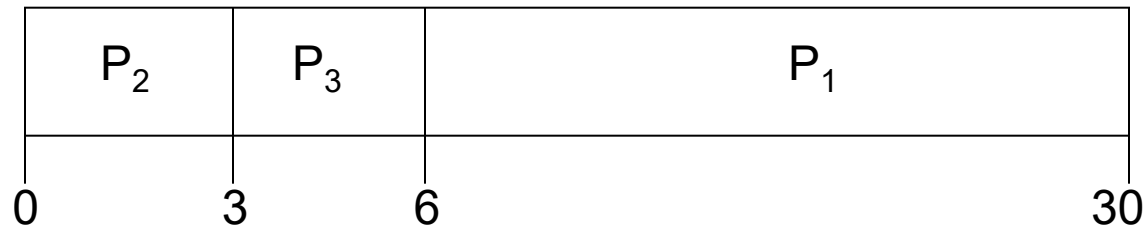
- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling (Cont)

Suppose that the processes arrive in the order

$$P_2, P_3, P_1$$

- The schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case
- *Convoy effect*: short process behind long process

Lesson from Previous Example

- What is an implication from the previous example?

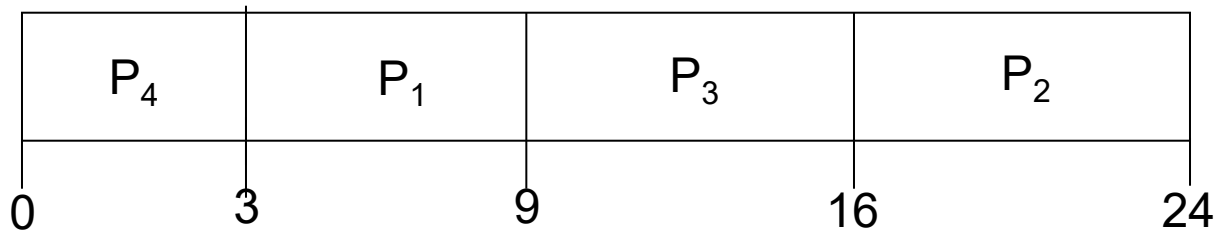
Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time

Example of SJF

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

- SJF scheduling chart



- Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$

Shortest-Job-First (SJF) Scheduling

- SJF is optimal
 - It gives minimum average waiting time for a given set of processes
 - The difficulty is how to know the length of the next CPU request

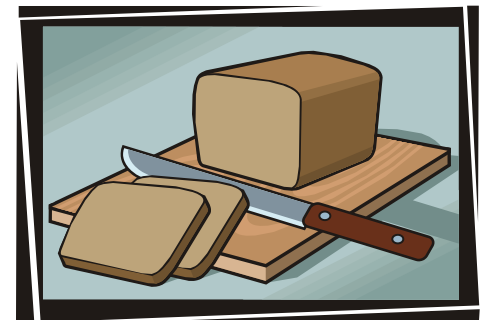


Issues to Think about

- SJF is good. But we might not know CPU burst time in advance. How can we handle this?

Round Robin (RR)

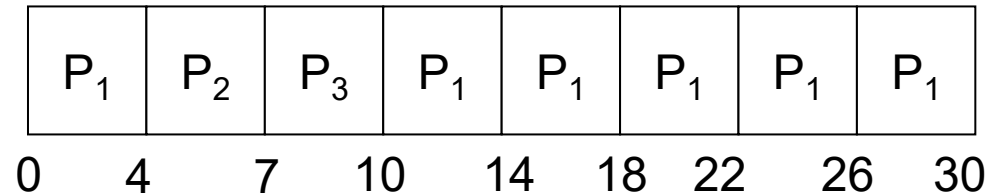
- Each process gets a small unit of CPU time (*time quantum*), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high



Example of RR with Time Quantum = 4

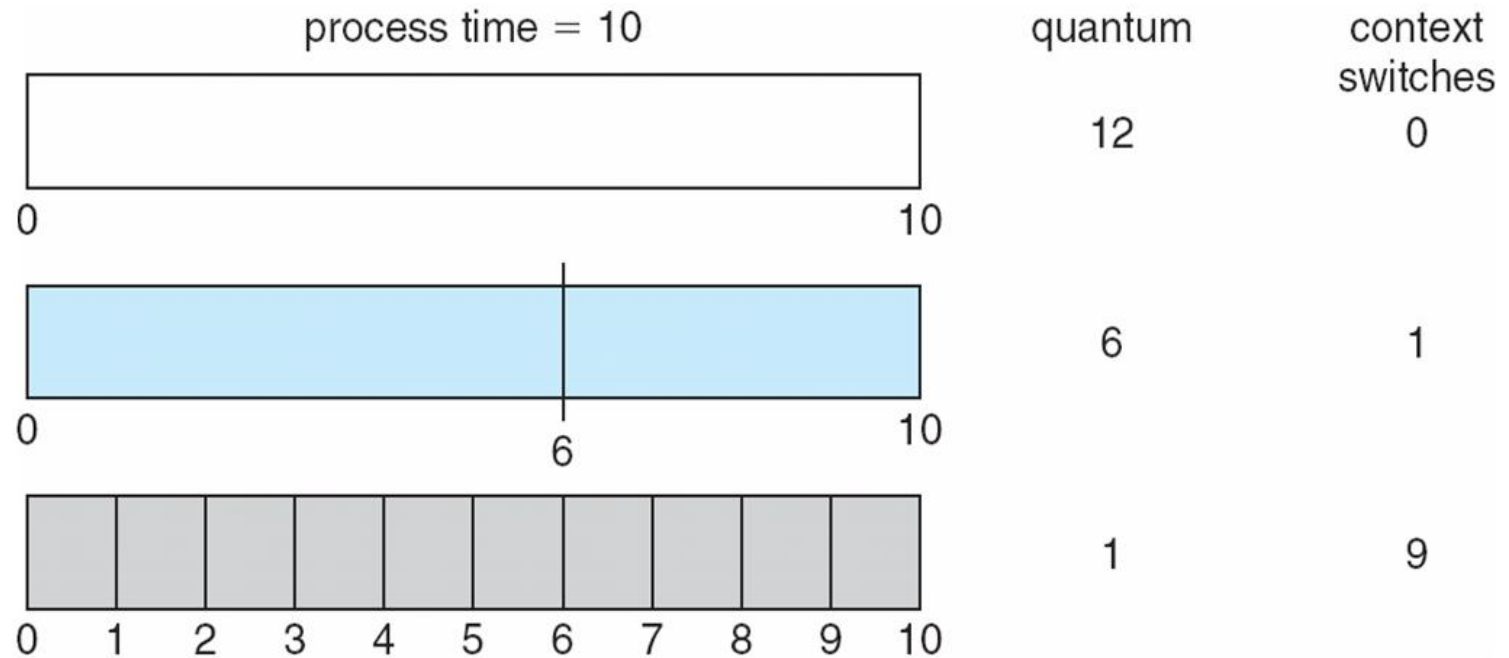
<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- The schedule is:



- Typically, higher average turnaround than SJF, but better *response*
 - Average waiting time = $(6+4+7)/3 = 5.66$

Time Quantum and Context Switch Time



Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
 - Preemptive
 - nonpreemptive
- SJF is a priority scheduling where priority is the predicted next CPU burst time
- Problem $\equiv$ **Starvation**
 - low priority processes may never execute
- Solution $\equiv$ **Aging**
 - as time progresses, it increases the priority of the process

Multilevel Queue

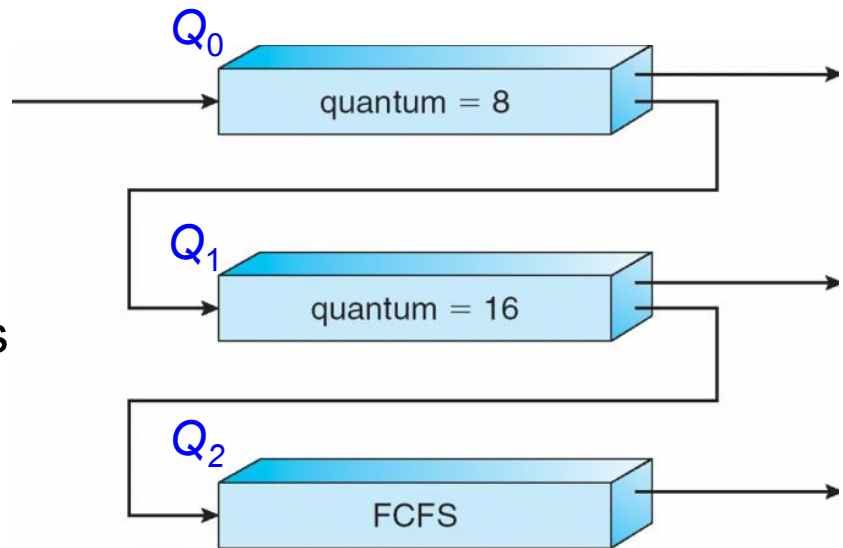
- Ready queue is partitioned into separate queues:
 - foreground (interactive), background (batch)
- Each queue has its own scheduling algorithm
 - foreground – FCFS
 - background – RR

Example: three queues:

Q_0 – FCFS time quantum 8 ms

Q_1 – FCFS time quantum 16 ms

Q_2 – RR



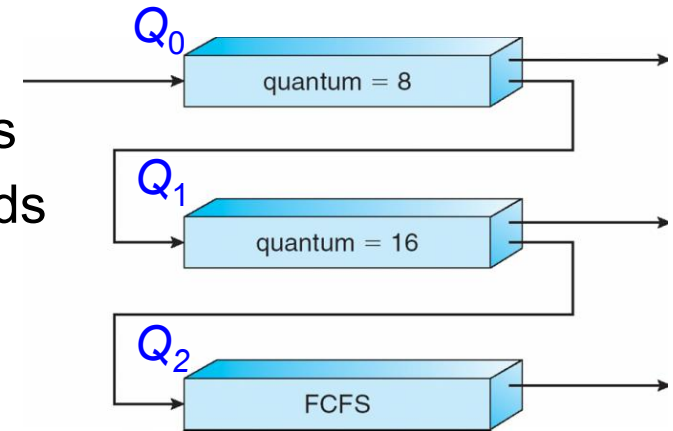
Multilevel Queue

- Ready queue is partitioned into separate queues:
 - foreground (interactive), background (batch)
- Each queue has its own scheduling algorithm
 - foreground – FCFS
 - background – RR
- Scheduling must be done between the queues, for examples,
 - Fixed priority scheduling (i.e., serve all from foreground then from background). Possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in FCFS, 20% to background in RR

Example of Multilevel Feedback Queue

- Three queues:

- Q_0 – FCFS time quantum 8 milliseconds
- Q_1 – FCFS time quantum 16 milliseconds
- Q_2 – RR



- Scheduling

- A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1 .
- At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2 .

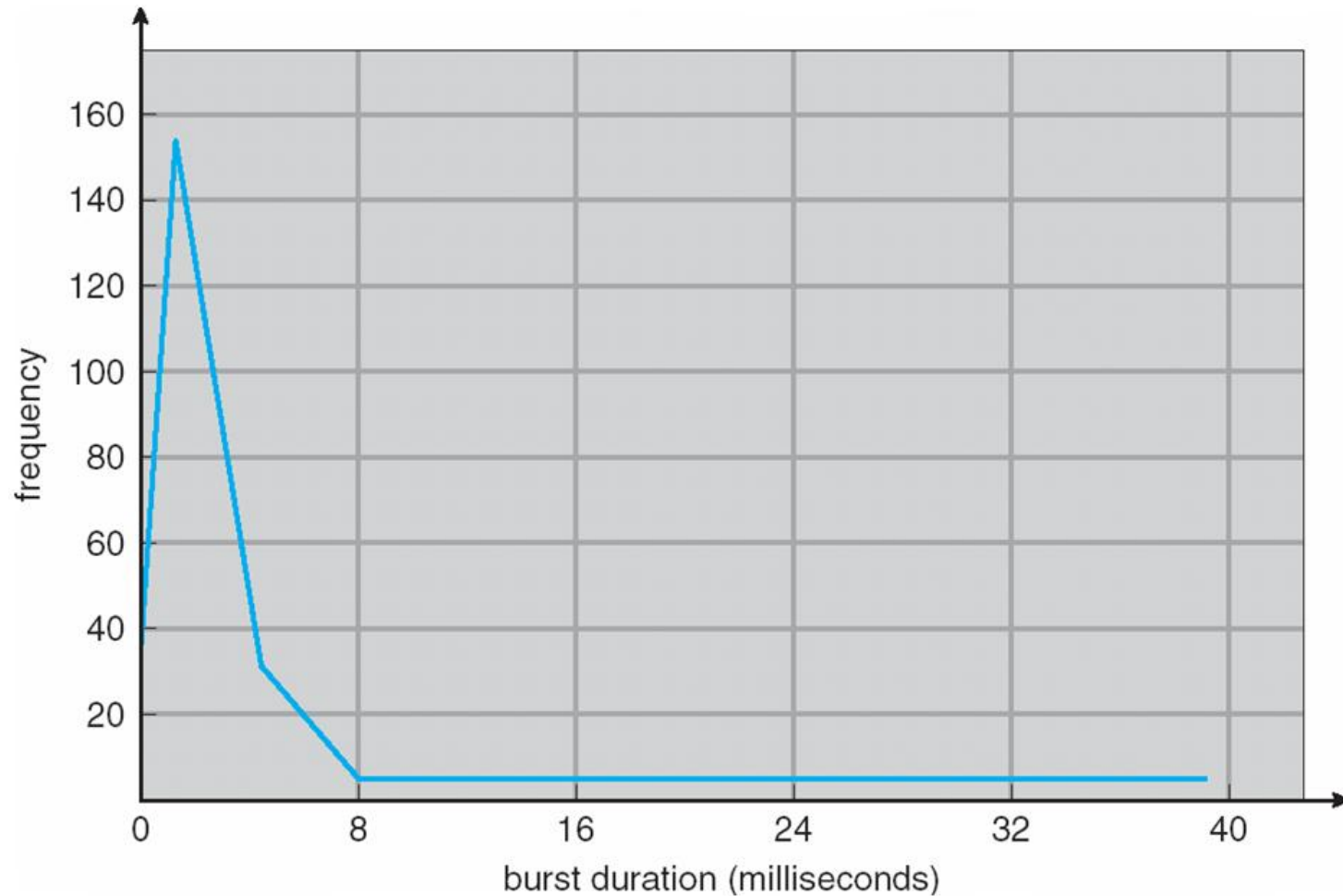
Issues

- What kinds of processes does multilevel feedback queue favor?

Issues

- What kinds of processes does multilevel feedback queue favor?
 - Short CPU bursts
 - I/O bound (CPU bound will go to lower-priority queue)
 - Interactive

Histogram of CPU-burst Times



Multiple-Processor Scheduling

- Must decide more than which process to run
 - must decide on which CPU to run it
- Moving between CPUs has cost
 - more cache misses, depending on architecture more TLB misses too
- *Affinity* scheduling
 - try to keep processes on the same CPU
 - but also prevent load imbalances
 - do cost-benefit analysis when deciding to migrate

Priority Inversion

- What is it?
- How can we solve it?

Priority in Scheduling

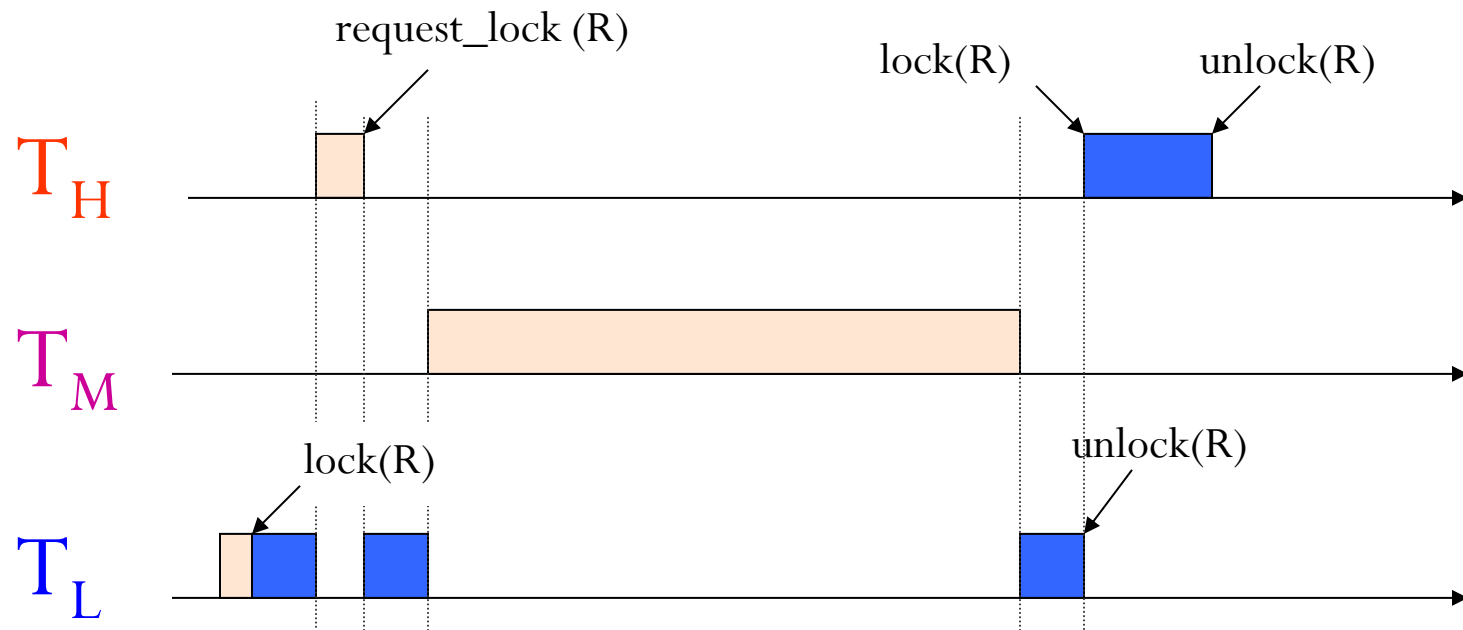
- Priority represents importance or urgency
 - A high priority represents that it is more likely to be subject to strict time constraints of corresponding jobs — it may be providing data to an interactive user, or acting subject to real-time response guarantees.
 - A low priority represents that it is not important for a corresponding job to finish promptly (for example, they might be a batch job or another non-interactive activity).
- Priority scheduling favors the execution of processes with higher priorities

Priority Inversion

- Priority inversion
 - a scenario where a lower-priority task (unnecessarily) blocks the execution of a higher-priority task
 - negates the principle of priority-based scheduling
- How it happens
 - a lower-priority task T_L holds a share resource R_k
 - a medium-priority task T_M preempts T_L
 - a higher-priority task T_H wants to access R_k
 - Then, T_H should wait for T_M to finish, and then for T_L to release R_k
 - This effectively "inverting" the relative priorities of the two tasks.

Priority Inversion

□ normal execution □ execution within critical section



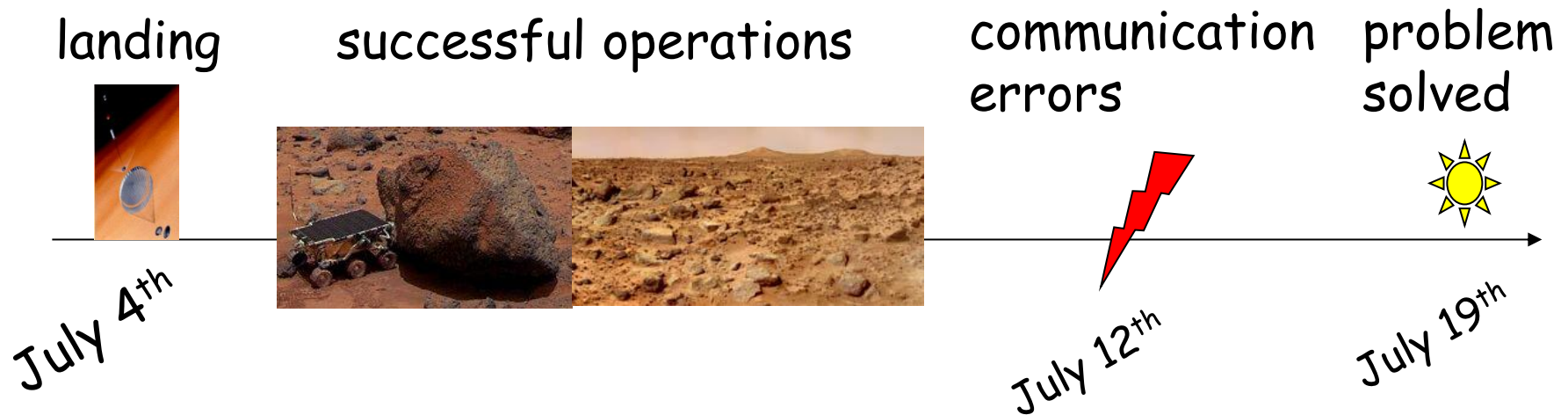
Priority Inversion

- It's fine, most of time
 - In some cases, priority inversion can occur without causing immediate harm—the delayed execution of the high priority task goes unnoticed, and eventually the low priority task releases the shared resource.
- However, it can be very critical at some point
 - If the high priority task is left starved of the resources, it might lead to a system malfunction or the triggering of pre-defined corrective measures, such as a watch dog timer resetting the entire system.
- Good example
 - The trouble experienced by the Mars lander “Mars Pathfinder” is a classic example of problems caused by priority inversion

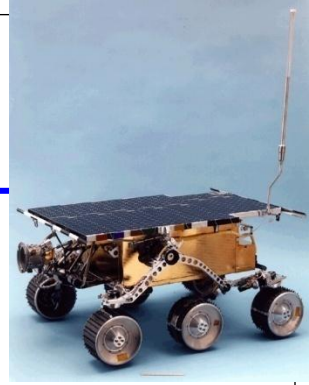
Priority Inversion & Mars Pathfinder



- Spacecraft landed on the Mars on July 4th, 1997 for the mission operations, including
 - surface operations
 - daily images of Mars
 - daily weather reports from the surface of Mars
- Out-of-control a week after landing, due to software problem

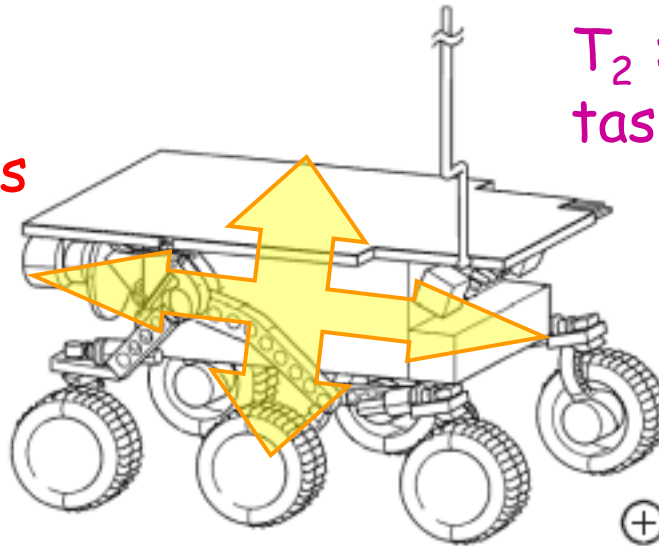


Mars Pathfinder Configuration



- VxWorks
 - real-time embedded system kernel
 - preemptive priority-based scheduler
- Information Bus
 - shared memory thru synchronized access

T_1 : information bus
management
task (hp)

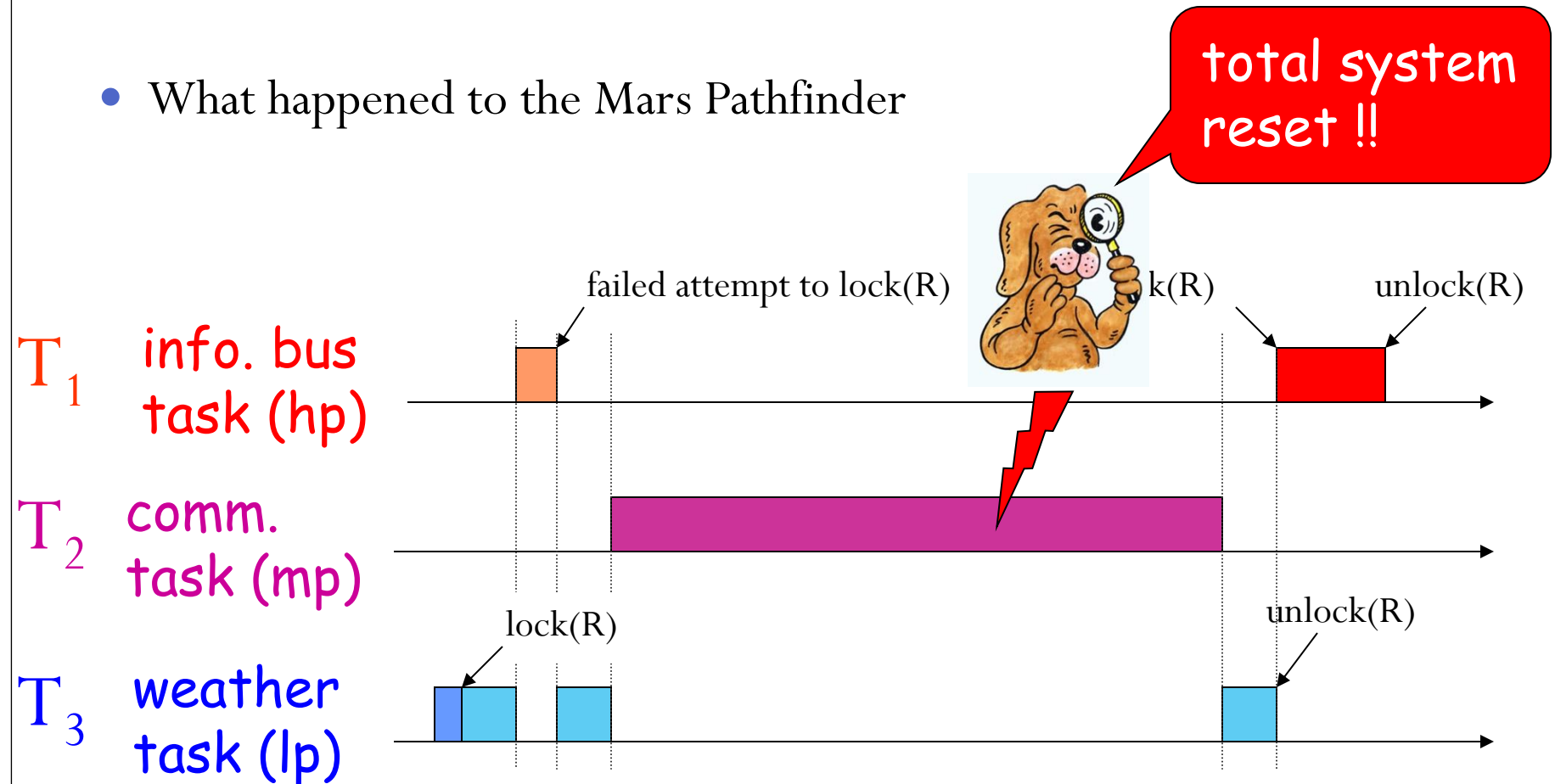


T_2 : communication
task (mp)

T_3 : weather
data gathering
task (lp)

The Priority Inversion Problem

- What happened to the Mars Pathfinder



The comm. task causes the info. bus task to wait

Solutions to Priority Inversion

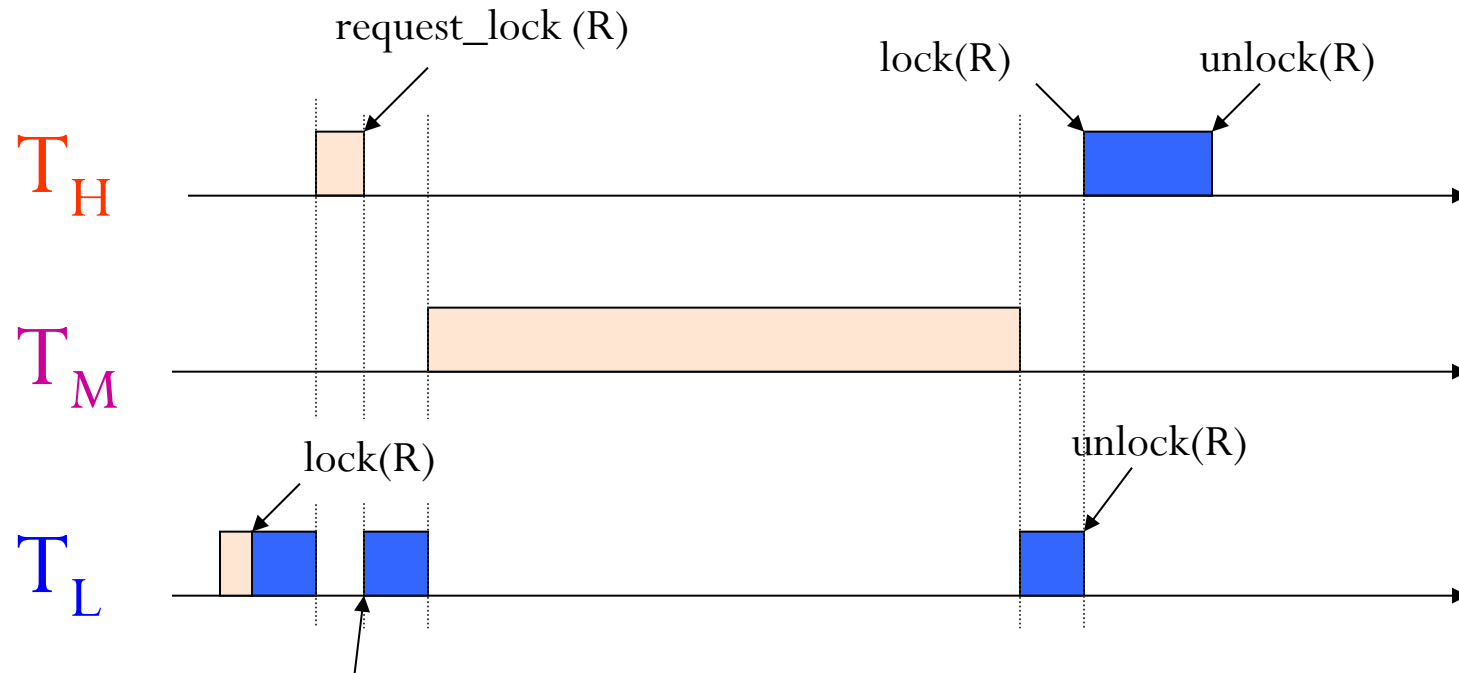
- Synchronization protocols
 - Priority inheritance protocol (PIP)
 - Priority ceiling protocol (PCP)
 - Stack resource protocol (SRP)

Priority Inheritance Protocol

- How it works
 - a lower-priority job T_L is accessing a shared resource R_k
 - a higher-priority job T_H wants access R_k
 - then, the priority of T_L increases to the one equivalent to the priority of T_H
 - after T_L executing its critical section, the priority of T_L returns back to its original priority level
- Lui Sha, Raj Rajkumar, John Lehoczky, “Priority inheritance protocols: An approach to real-time synchronization”, IEEE Trans. on Computers, 1990

Priority Inversion

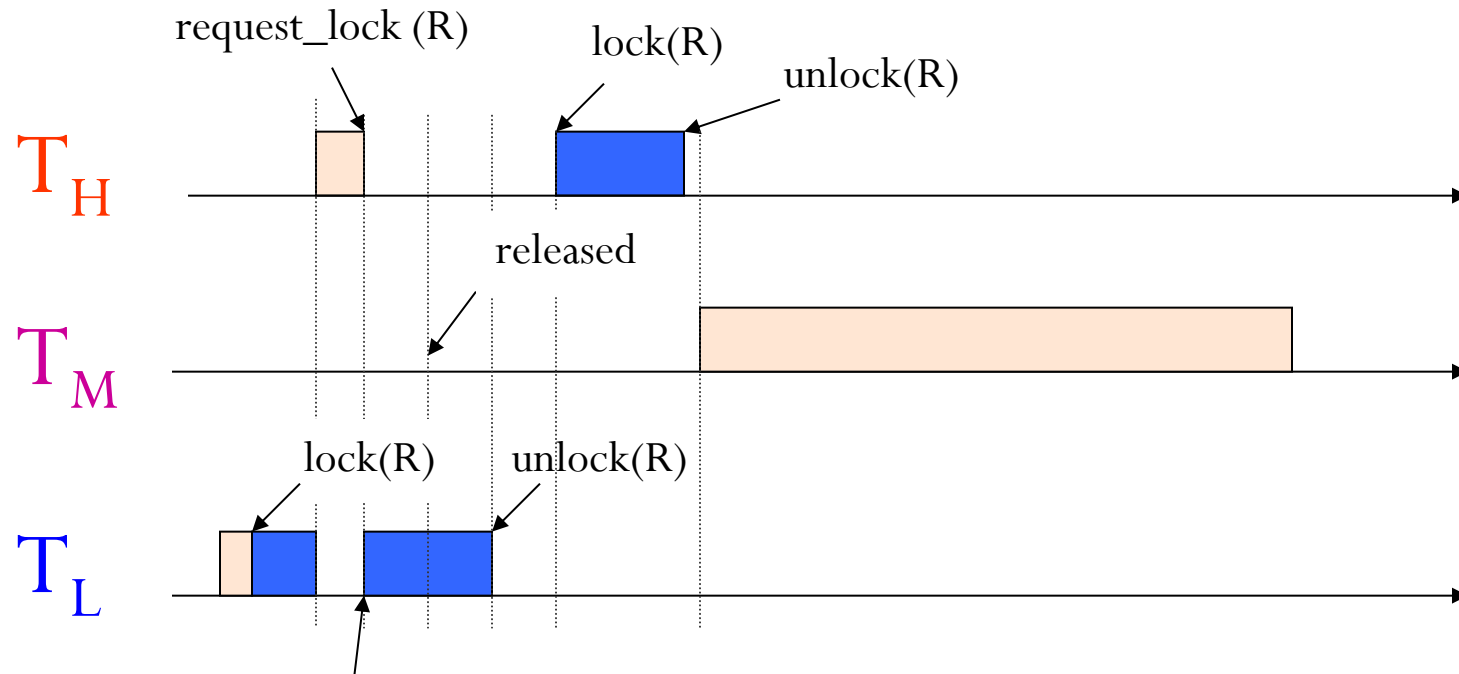
normal execution execution within critical section



Priority of T_L becomes equal to that of T_H

Priority Inheritance

normal execution execution within critical section



Priority of T_L becomes equal to that of T_H